

Actividad 4. Estándares y Visualización con Meta AI.

1. Si quiero mostrar la distribución de una variable en una y detectar valores atípicos, ¿por qué es mejor usar un Boxplot que un gráfico de barras tradicional? Explica las ventajas técnicas.

1. Qué mide cada uno

	Gráfico de barras	Boxplot
Diseñado para	Comparar totales, promedios o frecuencias entre categorías	Resumir la distribución completa de una variable numérica
Eje Y representa	Una agregación: suma, conteo, media	La escala original de los datos
Muestra 1 sola variable numérica	No, necesita categorías + valor agregado	Sí, es su especialidad

Por eso, si metes una variable numérica en barras, usualmente solo verías 1 barra con la media. Pierdes toda la forma de los datos.

2. Ventajas técnicas del boxplot para distribución + atípicos

1. Te da 5 números resumen de un vistazo

Un boxplot codifica visualmente:

- **Mínimo:** extremo del bigote inferior, sin contar atípicos
- **Q1:** percentil 25%, borde inferior de la caja
- **Mediana:** percentil 50%, línea dentro de la caja
- **Q3:** percentil 75%, borde superior de la caja

- **Máximo:** extremo del bigote superior, sin contar atípicos

Con eso ya sabes centro, dispersión y rango. Un gráfico de barras con la media te da 1 solo número. Si tus datos son sesgados, la media miente y la mediana del boxplot no.

2. Muestra la asimetría y dispersión

Si la mediana no está al centro de la caja, sabes que hay sesgo. Si una caja es mucho más larga que la otra, hay más dispersión de ese lado.

Una barra no te dice nada de eso. Dos conjuntos pueden tener la misma media pero distribuciones totalmente distintas, y la barra los mostraría idénticos.

3. Detecta atípicos con una regla estándar

El boxplot define atípicos automáticamente usando el método de Tukey:

$$RIC = Q3 - Q1$$

Cualquier punto que esté a más de

$$1.5 \times RIC$$

por debajo de $Q1$ o por encima de $Q3$ se dibuja como punto individual fuera de los bigotes.

Eso te da detección visual inmediata. En un gráfico de barras los atípicos se diluyen en la media y son invisibles. Un solo valor de 1,000,000 puede jalar toda la barra hacia arriba y nunca te enteras.

4. No asume normalidad

La media y desviación estándar que suelen acompañar a las barras asumen distribución simétrica. El boxplot usa cuartiles, así que funciona bien con datos sesgados, multimodales o con colas pesadas. Es no-paramétrico.

5. Compacto para comparar grupos

Si quieres comparar la distribución de "edad" entre 5 países, con 5 boxplots lado a lado ves centro, dispersión y atípicos de todos a la vez. Con barras solo compararías 5 medias, perdiendo todo lo demás.

Cuándo sí usar barras

Las barras son mejores cuando tu variable X es categórica y quieres comparar totales o frecuencias. Ej: "Ventas por producto", "Votos por candidato". Ahí no hay distribución que mostrar.

Regla práctica

Si tu pregunta es "¿cuánto hay en cada categoría?" → barras.

Si tu pregunta es "¿cómo se comporta esta variable y qué valores raros tiene?" → boxplot.

2. ¿Cuáles son las principales diferencias entre Matplotlib y Seaborn en términos de sintaxis y facilidad de uso para realizar un análisis de correlación bivariable?

1. Filosofía general

	Matplotlib	Seaborn
Nivel	Bajo nivel. Tú controlas cada elemento: ejes, colores, ticks, leyendas	Alto nivel. Construido sobre Matplotlib, con "defaults" estadísticos listos
Enfoque	Dibujar. Es una librería de plotting pura	Explorar. Es una librería estadística con plotting integrado
Curva de aprendizaje	Más empinada al inicio. Más código para gráficos simples	Más rápida para EDA. Menos código para gráficos estadísticos

2. Sintaxis para correlación bivariable

Para correlación bivariable lo más común es:

1. Scatter plot de las 2 variables
2. Heatmap de matriz de correlación
3. Regresión + intervalo de confianza

Caso 1: Scatter plot básico

Supongamos un DataFrame `df` con columnas `x` e `y`.

Matplotlib

Python

```
import matplotlib.pyplot as plt
plt.figure(figsize=(6,4))plt.scatter(df['x'], df['y'], alpha=0.6)plt.xlabel('Variable X')plt.ylabel('Variable Y') plt.title('Correlación X vs Y')plt.grid(True)plt.show()
```

4 líneas ocultas

Tú le pasas arrays. Si quieres color por categoría, tamaño por otra variable, tienes que hacerlo manual.

Seaborn

Python

```
import seaborn as sns
sns.scatterplot(data=df, x='x', y='y', alpha=0.6)plt.title('Correlación X vs Y') # plt sigue funcionando para ajustes
```

Seaborn entiende DataFrames directo. `data=df` + nombre de columna. Mucho menos boilerplate.

Caso 2: Scatter + línea de regresión

Matplotlib

No tiene regresión integrada. Tienes que calcular tú con `numpy` o `scipy` y luego graficar la línea:

Python

```
import numpy as npplt.scatter(df['x'], df['y'])m, b = np.polyfit(df['x'], df['y'], 1)plt.plot(df['x'], m*df['x'] + b, color='red')
```

Seaborn

Una línea:

Python

```
sns.regplot(data=df, x='x', y='y', ci=95) # ci = intervalo de confianza
```

Te da el scatter, la línea de regresión y el IC del 95% automático.

Caso 3: Matriz de correlación como heatmap

Matplotlib

Python

```
corr = df.corr()plt.imshow(corr, cmap='coolwarm')plt.colorbar()plt.xticks(range(len(corr)), corr.columns, rotation=45)plt.yticks(range(len(corr)), corr.columns)# Para poner los números tienes que hacer loop manual
```

1 línea oculta

Seaborn

Python

```
corr = df.corr()sns.heatmap(corr, annot=True, cmap='coolwarm', fmt='.2f')
```

`annot=True` pone los valores en cada celda. `fmt` controla decimales. 1 línea vs 6+ líneas.

3. Facilidad de uso para correlación bivariable

Tarea	Matplotlib	Seaborn	Ganador en facilidad
Scatter simple	<code>plt.scatter(x, y)</code>	<code>sns.scatterplot(data=df, x='x', y='y')</code>	Empate, pero Seaborn mejor con DataFrames
Color por categoría	Loop manual o <code>c=df['cat']</code> + colormap	<code>hue='categoria'</code>	Seaborn
Regresión + IC	Calcular y graficar tú	<code>sns.regplot()</code> o <code>sns.lmplot()</code>	Seaborn
Distribuciones marginales	Crear subplots manuales	<code>sns.jointplot()</code>	Seaborn
Heatmap anotado	6-8 líneas de código	<code>sns.heatmap(annot=True)</code>	Seaborn

Tarea	Matplotlib	Seaborn	Ganador en facilidad
Personalización total	Control de cada píxel	Limitado, hay que bajar a Matplotlib	Matplotlib
Estética por defecto	Bastante básica	Paletas y estilos pro desde el inicio	Seaborn

`jointplot` merece mención aparte para correlación bivariable:

Python

```
sns.jointplot(data=df, x='x', y='y', kind='reg', marginal_kws=dict(bins=20))
```

Te da scatter + regresión al centro, y histogramas/KDE de cada variable en los márgenes. Hacer eso en Matplotlib puro son 20+ líneas.

4. ¿Cuándo usar cuál?

Usa Seaborn si:

1. Estás en fase exploratoria/EDA y quieres ver correlaciones rápido
2. Trabajas con `pandas.DataFrame` y quieres pasar nombres de columnas directo
3. Necesitas estadística visual: regresión, IC, distribuciones, categorías con `hue`
4. Quieres gráficos bonitos con poco código

Usa Matplotlib si:

1. Necesitas control pixel-perfect para publicación
2. Vas a hacer un gráfico muy custom que Seaborn no cubre
3. No quieres otra dependencia
4. Ya tienes todo el pipeline en Matplotlib

En la práctica: la mayoría usa ambos. Seaborn para el 80% del EDA porque es rapidísimo, y luego baja a Matplotlib para ajustar detalles: `fig, ax = plt.subplots()`, `sns.regplot(ax=ax, ...)` y después `ax.set_xlim()`, `ax.set_title()`, etc.

Seaborn está construido encima de Matplotlib, así que no son excluyentes. Todo gráfico de Seaborn devuelve objetos de Matplotlib que puedes seguir editando.

3. Dame 3 ejemplos de cómo se pueden 'manipular' visualmente los datos para engañar a una audiencia en una presentación y cómo puedo evitar cometer estos errores en mi proyecto de Data Science.

1. Truncar el eje Y para exagerar diferencias

Cómo manipula: Si empiezas el eje Y en 15 en lugar de 0, una diferencia de 15 a 16 se ve gigantesca. Se usa mucho en noticias y política para que un cambio mínimo parezca dramático.

Ejemplo engañoso:

- Empresa A: 15.2 ventas
- Empresa B: 15.8 ventas Con el eje Y de 15 a 16, la barra de B se ve 4 veces más alta que la de A, aunque solo es 4% más.

Cómo evitarlo:

- **Regla base:** Para gráficos de barras, siempre empieza el eje Y en 0. Las barras representan longitud, y cortar el eje rompe la proporción visual.
- **Excepción válida:** En gráficos de línea donde te importa la tendencia, no la magnitud total, puedes hacer zoom. Pero marca claramente el corte con  en el eje y anótalo: "Eje Y inicia en 15 para mostrar variación".

2. Usar escalas o bins engañosos

Cómo manipula: Dos trucos comunes acá:

1. **Escalas no lineales sin avisar:** Usar escala logarítmica para aplanar picos o lineal para ocultar crecimiento exponencial.
2. **Histogramas con bins raros:** Agrupar datos en intervalos desiguales. Ej: bins de [0-10], [10-50], [50-100] hace que el bin del medio se vea masivo aunque tenga pocos datos por unidad.

Cómo evitarlo:

- **Elige la escala por la data, no por la historia:** Usa lineal para diferencias absolutas. Usa log solo si tus datos abarcan varios órdenes de magnitud y dilo explícito: "Escala logarítmica".
- **Bins uniformes:** En histogramas, usa intervalos de igual tamaño. Si necesitas bins desiguales por negocio, grafica densidad en lugar de conteo para que la altura sea comparable.
- **Compara manzanas con manzanas:** Si pones 2 líneas en un gráfico con doble eje Y, asegúrate que las escalas tengan sentido entre sí. Mejor aún: usa gráficos separados.

3. Cherry-picking de datos o tiempo + 3D/distorsión

Cómo manipula:

1. **Recorte temporal:** Mostrar solo 2023-2024 donde tus ventas subieron, omitiendo que 2020-2022 fueron en picada.
2. **Efectos 3D y perspectiva:** Un gráfico de pastel en 3D hace que las rebanadas del frente se vean 2x más grandes. La inclinación distorsiona ángulos y áreas.

Cómo evitarlo:

- **Muestra el contexto completo:** Incluye series temporales largas. Si haces zoom a un periodo, pon un mini-gráfico arriba con la serie completa para referencia.
- **Di NO al 3D:** Excel lo ama, pero en Data Science el 3D solo añade distorsión. Usa 2D siempre. Los gráficos de pastel ya son malos para comparar; si tienes >3 categorías, usa barras horizontales.
- **Documenta tus filtros:** En el pie del gráfico pon "Datos de Q1 2024, n=1,200. Se excluyeron outliers >3 std". Transparencia mata sesgo.

Reglas de oro para tu proyecto

Mala práctica	Qué hacer en su lugar
Eje Y truncado en barras	Eje Y siempre en 0 para barras

Mala práctica	Qué hacer en su lugar
Gráfico 3D con perspectiva	Gráfico 2D plano, mismo plano visual
Omitir contexto temporal	Mostrar serie completa + anotar eventos clave
Colores semáforo arbitrarios	Usa color para destacar, no para engañar. Rojo=malo solo si de verdad es malo
Eje dual sin relación	Mejor dos gráficos separados, alineados verticalmente

Reflexión.

Las rutinas de graficación como método exploratorio de información deben tomarse con cautela pues visualmente pueden ser descriptivas pero el verdadero comportamiento de los datos puede desvelar características interesantes cuando comienza el análisis numérico.

En cuestión de la visualización de gráficas ajenas siempre hay que hallarse alerta en las escalas pues, como menciona Meta AI, existen varias maneras para hacer "trampa" y mostrar datos bonitos que busquen despistar al espectador. De igual manera, la experiencia con el análisis de gráficos es lo que nos va a permitir tener una intuición sobre los datos lo suficientemente buena para comprender su comportamiento sin aplicar formulas.